Documind – Multi-Document Conversational AI

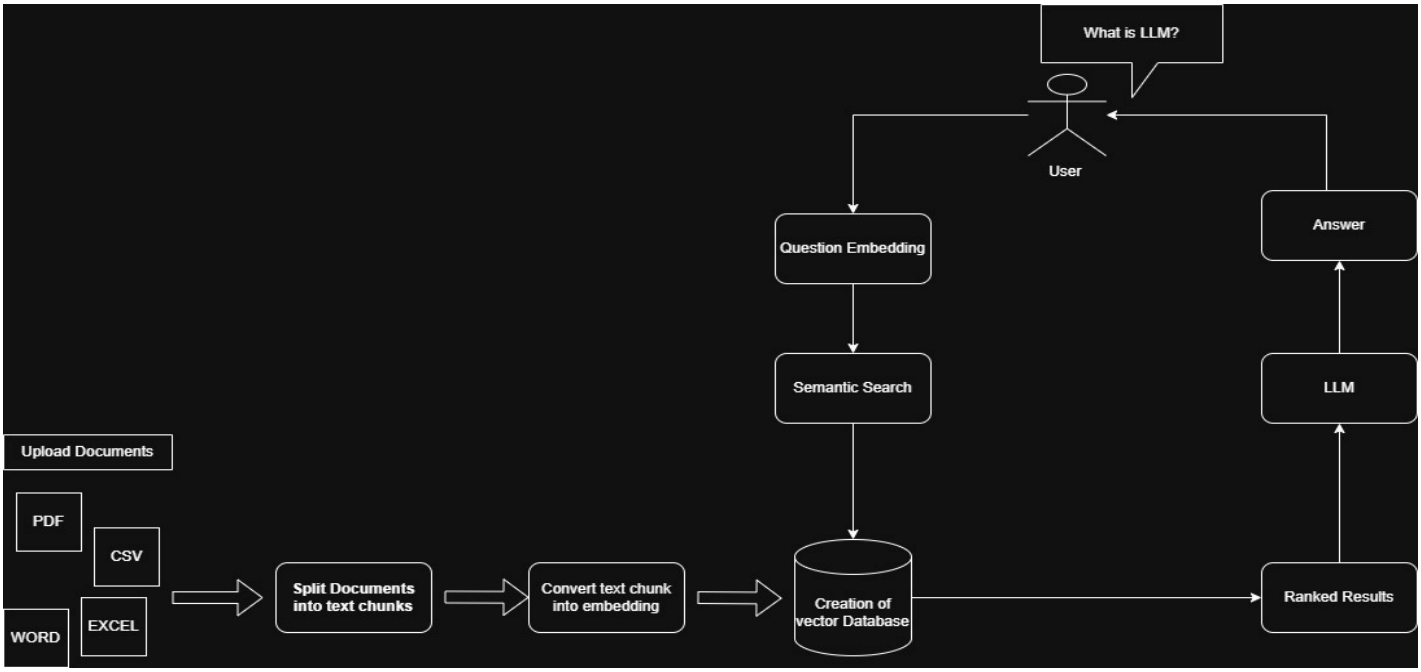
1. Problem Statement

In today's digital environment, users often deal with large volumes of unstructured documents such as PDFs, Word files, CSVs, and Excel sheets. Extracting relevant information manually is time-consuming and inefficient. Traditional search systems rely on keyword matching and fail to understand context, leading to inaccurate or incomplete results.

The problem addressed in this project is to build an intelligent system that allows users to interact with multiple documents using natural language queries and retrieve precise, context-aware answers efficiently.

2. Methodology

The Documind system is built using a **Retrieval-Augmented Generation (RAG)** approach, combining semantic search with a local Large Language Model (LLM).



Workflow:

1. **Document Ingestion**
   - Supports PDF, DOCX, CSV, and Excel files.
   - Files are loaded and converted into text.
2. **Text Splitting**
   - Documents are divided into smaller chunks using Recursive Character Text Splitter.
   - Ensures better context handling and retrieval accuracy.
3. **Embedding Generation**
   - Each chunk is converted into vector embeddings using embedding models.
   - These embeddings capture semantic meaning.
4. **Vector Storage (FAISS)**
   - Embeddings are stored in a FAISS vector database for efficient similarity search.

5. **Query Processing**
    - User query is converted into embedding.
    - Similar document chunks are retrieved using FAISS.
6. **Response Generation**
    - Retrieved context is passed to a local LLM (via Ollama).
    - Model generates a contextual answer.
7. **Source Highlighting**
    - Displays document name and page number.
    - Enables explainability and transparency.

3. Prompt Design

The system uses a structured prompt to guide the LLM:

- Ensures answers are based on retrieved document context.
- Handles fallback for general queries.
- Avoids hallucination by instructing the model to rely on context.

Prompt Template:

- Provide context from retrieved chunks.
- Ask the model to answer using context.
- Allow normal conversational response if query is general.

This design ensures:

- Accurate responses for document-based queries.
- Natural conversation for casual inputs.

4. Results & Comparison

Performance Observations:

- Accurate responses for document-specific queries.
- Fast retrieval using FAISS indexing.
- Efficient handling of multiple document formats.

Comparison with Traditional Systems:

Feature	Traditional Search	Documind
Search Type	Keyword-based	Semantic
Context Understanding	Limited	High
Response Generation	None	LLM-based
Multi-document Support	Limited	Yes
Explainability	No	Yes (Source Highlighting)

5. Observations & Insights

- **Semantic search significantly improves accuracy** compared to keyword-based systems.
- **Chunk size and overlap impact performance**:
  - Smaller chunks → faster processing.
  - Balanced size → better context.
- **Local LLMs reduce dependency on external APIs**, enabling offline functionality.
- **Voice input enhances user interaction**, making the system more accessible.
- **Source highlighting improves trust and transparency** in AI-generated responses.
- System performance depends on:
  - Available RAM
  - Model size (phi3, mistral, etc.)
  - Number of retrieved chunks

Conclusion

Documind successfully demonstrates an intelligent document interaction system using RAG architecture. It provides accurate, context-aware answers, supports multiple document formats, and enhances usability through features like voice input and source highlighting. The project highlights the practical application of LLMs in real-world information retrieval tasks.